

Image-Based Roadmaps for Vision-Only Planning and Control of Robotic Manipulators

Sreejani Chatterjee^{ID}, *Graduate Student Member, IEEE*, Abhinav Gandhi^{ID}, Berk Calli^{ID}, *Member, IEEE*, and Constantinos Chamzas^{ID}

Abstract—This work presents a motion planning framework for robotic manipulators that computes collision-free paths directly in image space. The generated paths can then be tracked using vision-based control, eliminating the need for an explicit robot model or proprioceptive sensing. At the core of our approach is the construction of a roadmap entirely in image space. To achieve this, we explicitly define sampling, nearest-neighbor selection, and collision checking based on visual features rather than geometric models. We first collect a set of image space samples by moving the robot within its workspace, capturing keypoints along its body at different configurations. These samples serve as nodes in the roadmap, which we construct using either learned or predefined distance metrics. At runtime, the roadmap generates collision-free paths directly in image space, removing the need for a robot model or joint encoders. We validate our approach through an experimental study in which a robotic arm follows planned paths using an adaptive vision-based control scheme to avoid obstacles. The results show that paths generated with the learned-distance roadmap achieved 100% success in control convergence, whereas the predefined image space distance roadmap enabled faster transient responses but had a lower success rate in convergence.

Index Terms—Motion and path planning, collision avoidance, integrated planning and control.

I. INTRODUCTION

VISION-BASED control techniques [1], [2], offer significant advantages for robotic manipulators in unstructured and cluttered environments by enabling closed-loop control using task-relevant visual information. These techniques also enhance robustness against model inaccuracies, beneficial for robots with complex or variable dynamics [3], [4], [5]. This strategy is especially useful for robots that are difficult to model accurately, e.g. soft robots [6], [7], under-actuated robots [8], [9], 3D printed robots [10], [11], and robots with inexpensive hardware [12]. Furthermore, model-free visual servoing, which learns robot-feature motion models during control, reduces reliance on a priori knowledge of the robot model [13], [14].

The goal of our research is to push the boundaries of purely vision-based control and motion planning for robotic manipulators, by decreasing reliance on explicit robot modeling or

proprioceptive sensing. In doing so, we strive to use natural visual features along the robot's body in image space, without attaching any external markers. We use multiple features along the robot's body to uniquely distinguish full-body configurations that have the same end effector position, a characteristic common in redundant robots such as continuum manipulators. While the works in [15], [16], [17] provided algorithms to track natural keypoints and use them to achieve vision-based model-free control with decent transient responses, these algorithms are only designed to run in obstacle-free space and do not provide motion planning capability to avoid obstacles in the scene.

In this work, we tackle the problem of collision-free path planning for purely vision-based robot control: we develop a planning and control scheme that does not rely on a robot model or on-board sensors (e.g. joint encoders) in runtime. As such, this strategy is especially useful for (but not limited to) soft/underactuated/inexpensive robots that are hard to model and may not have reliable proprioceptive sensing.

Toward this goal, we propose a novel motion planning formulation that operates only with visual information. We propose a vision-only motion planning methodology using roadmaps [18]. We investigated two ways of generating roadmaps using natural features on a robot: 1) directly utilizing the Euclidean distance between keypoints in image space as a distance metric, 2) estimating the joint displacements from image features and utilizing it as a distance metric. For the latter, we first used an automated data collection pipeline to annotate natural keypoints' placement in image space along the robot's body as the robot moves across various configurations.

This dataset was used to develop a simple neural network that approximates joint displacements based on keypoint locations in image space. These distance metrics were integrated into the roadmap construction. Once the roadmaps are constructed, polygon-based collision checking and A* [19] search are employed to ensure collision-free paths. These two approaches have different implications for vision-based robot control. In a nutshell, we observed that utilizing estimated joint distances in the roadmap results in smoother and more accurate tracking of the generated path, allowing the robot to stay in its defined workspace and avoid obstacles. In contrast, roadmaps based on Euclidean distances in image space can offer faster transient responses, albeit with potentially less reliable tracking. In environments with tightly spaced or irregularly shaped obstacles, these image space roadmaps frequently fail to generate feasible paths due to their poor alignment with the underlying joint-space feasibility. Our experiments demonstrate these aspects both in the presence and absence of obstacles.

Received 11 February 2025; accepted 13 June 2025. Date of publication 3 July 2025; date of current version 15 July 2025. This work was supported by the National Science Foundation under Award 2341532. This article was recommended for publication by Associate Editor T. Do and Editor A. Bera upon evaluation of the reviewers' comments. (*Corresponding author: Sreejani Chatterjee.*)

The authors are with the Robotics Engineering Department, Worcester Polytechnic Institute, Worcester, MA 01609 USA (e-mail: schatterjee@wpi.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2025.3585760>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3585760

II. RELATED WORK

Motion planning is a core problem in robotics that has been extensively studied over the decades. It is generally categorized into three main types: optimization-based [20], [21], sampling-based [22], [23], and search-based planners [24], [25]. Recently, Jacobian-based motion planning [26] has also gained traction for obstacle avoidance tasks. While all methods have found widespread success in different applications, all of these rely on having an explicit geometric model of the robot to design feasible paths. In this work we extend the principles of sampling-based planning of the Probabilistic Roadmap (PRM) approach [18], to provide a method for motion planning where a robot model is not available.

Model-free planning, especially without prior knowledge of the robot's geometry, presents unique challenges. Reinforcement learning (RL) has been explored extensively in the recent decades to address such challenges. For instance, [27] proposed an RL-based framework for generating jerk-free, smooth trajectories. While effective, this method depends on carefully crafted reward functions and extensive datasets generated in simulation, with no real-world validation. Similarly, [28] introduced a hybrid approach combining RRT*-based trajectories with PPO reinforcement learning for policy refinement. However, this method heavily relies on model-based elements like precomputed trajectories and supervised learning for initial policy design, with experiments confined to simulated environments. In contrast, our approach eliminates reliance on explicit or precomputed models and trajectories by leveraging visual keypoints to construct roadmaps directly in image space, requiring only a small dataset collected from a real robot. This makes our method more adaptable to scenarios without precise geometric models.

Among related works, the approaches proposed in [29] and [30] are the most closely aligned with ours. In [29], the authors plan trajectories by learning a low-dimensional latent space using three neural networks: an autoencoder, a latent dynamics model, and a collision checking network. This latent space allows planning without directly operating in the high-dimensional input space (e.g., raw images or complex joint configurations). While effective, their method requires extensive simulation-based training, which introduces a significant sim-to-real gap. Moreover, the system is validated only in simulation, and the reliance on multiple learned modules increases complexity, making real-world deployment more challenging. In contrast, our method operates directly in image space using visually tracked keypoints, avoiding the need for encoding, decoding, or learned dynamics. It uses real robot data exclusively, both for training and evaluation, and does not rely on simulation. This results in a lightweight, practical framework that is easier to implement and deploy in real-world scenarios. In [30], the authors learn the topology of a perceptual control manifold (PCM) using a topology-preserving neural network (TRN) constructed over both robot joint configurations and corresponding image features. This approach explicitly relies on access to **joint space** information and was designed specifically for a pneumatically driven robotic arm, limiting its generalizability. In contrast, our method requires only a dataset of image features and does not depend on any explicit robot model or configuration information. As a result, our approach is more general and can be easily modified to both rigid and soft robotic manipulators.

Planning a path for image based visual servoing without a-priori knowledge of the robot's model is rarely delved into in the literature. For instance, [31] modeled a path planner

for visual servoing to bridge gaps between initial and target positions which are much further apart in configuration space, without addressing collision avoidance. In [32] authors achieve obstacle avoidance in pose based visual servoing and hence still needed explicit robot model instead of only visual feedback.

III. PRELIMINARIES AND PROBLEM STATEMENT

In this section we describe the motion planning problem, a brief review of sampling-based methods focusing on probabilistic roadmaps, and finally we introduce the problem statement that we are trying to solve.

A. Model-Based Motion Planning

Let $x \in \mathcal{X}$ denote the state and state-space, and $u \in \mathcal{U}$ the control and control-space of a robot system [22]. The system dynamics equations f of the robot system can be written as

$$x(\mathcal{T}) = x(0) + \int_0^{\mathcal{T}} f(x(t), u(t)) dt \quad (1)$$

If the system dynamics impose constraints on the allowable paths the system is called a non-holonomic system. Now, Let $\mathcal{X}_{\text{obs}} \subset \mathcal{X}$ denote the inadmissible state space, which is the set of states that violates the robots kinematic constraints or collides with the workspace $\mathcal{W} \subseteq \mathbb{R}^3$. e.g.,

$$\mathcal{X}_{\text{obs}} = \{x \in \mathcal{X} \mid R(x) \cap \mathcal{O} \neq \emptyset\} \quad (2)$$

where $R(x) \subseteq \mathbb{R}^3$ is the set of all points occupied by the robot at state x , and $\mathcal{O} \subset \mathcal{W}$ denotes the set of obstacles within the workspace.¹ Let $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$ denote the collision-free state space. Also, let $x_{\text{start}} \in \mathcal{X}_{\text{free}}$ and $\mathcal{X}_G \subseteq \mathcal{X}_{\text{free}}$ represent the start state and goal region, respectively.

Motion Planning Problem: Given the motion planning tuple $(\mathcal{X}, R, \mathcal{W}, \mathcal{U}, f, x_{\text{start}}, \mathcal{X}_G)$ find a time $\mathcal{T}$ and a set of controls $u : [0, \mathcal{T}] \rightarrow \mathcal{U}$ such that the motion described by (1) satisfies $x(0) = x_{\text{start}}$, $x(\mathcal{T}) \in \mathcal{X}_G$ and $x(t) \in \mathcal{X}_{\text{free}}$.

The problem we are considering in this setting is departing from the above motion planning problem as the geometric model $R(x)$, the dynamics f and the workspace $\mathcal{W}$ is not directly available. Instead the only information available is the space of admissible controls $\mathcal{U}$ and an image $Im \in \mathcal{IM}$.

We will describe the new problem statement by explicitly defining equivalent concepts of obstacles, robot states, and state dynamics directly in the image space. We assume that we are given a point tracking function that maps a given image to N fixed pixel points on the robot's body. This vector of pixel points is denoted as an image state $\mathbb{K} \in I$. Where $I \subset \mathbb{R}^{N \times 2}$ denotes the space of all image states. In Fig. 1 we can see 4 different robot configurations in image space. Each configuration is represented by a set of 5 same-colored circles. Each of these circles on the robot's body in the image is a pixel point and is denoted as a keypoint or $\mathbb{k}$. The kinematic chain or shape formed by the vector of 5 circles represents one image state $\mathbb{K}$.

Given a set of image obstacles $\mathcal{IO} \subset Im$ we define:

$$I_{\text{obs}} = \{\mathbb{K} \in I \mid RI(\mathbb{K}) \cap \mathcal{IO} \neq \emptyset\} \quad (3)$$

where $RI(\mathbb{K}) \subseteq Im$ is the set of pixels that the robot occupies at image state $\mathbb{K}$. Similarly to before, we define $I_{\text{free}} = I \setminus I_{\text{obs}}$, $\mathbb{K}_{\text{start}}$ and I_g .

¹This equation only encodes collisions; joint or kinematic constraints can be incorporated in a similar manner.

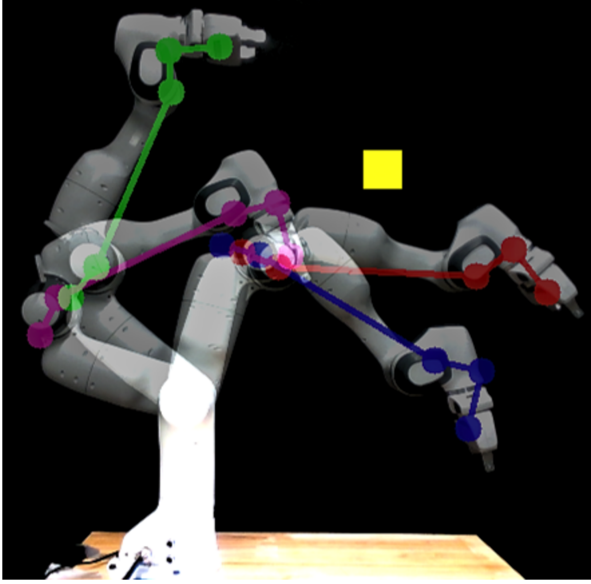


Fig. 1. The robot follows a collision-free path using visual keypoints (colored) planned with the proposed method. The set of keypoints are tracked in order: green, magenta, blue, and red. The robot avoids the yellow obstacle while moving between the specified start and goal.

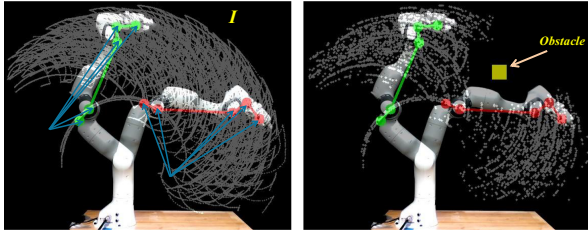


Fig. 2. The images depict a vision-based motion planning problem. The left image shows the discretized representation of I highlighted in gray, where each configuration is sampled as an image frame, resulting in a finite set of $\mathbb{K}$. The gray region in the right image highlights the same discretized representation of I_{free} , avoiding the obstacle in yellow. I_{free} ensures a collision-free path from the start configuration $\mathbb{K}_{\text{start}}$ to the goal configuration $\mathbb{K}_{\text{goal}}$.

We define the unknown system dynamics function as:

$$\mathbb{K}(\mathcal{T}) = \mathbb{K}(0) + \int_0^{\mathcal{T}} g(\mathbb{K}(t), u(t)) dt \quad (4)$$

Here we note that even if the underlying function f is fully-integrable (holonomic-system) if $\dim(I) > \dim(\mathcal{X})$ the equivalent system dynamics equations g will be non-holonomic as there will be paths in the higher dimensional image space I that cannot be followed by the robot. In our approach we consider this issue, and propose a way to produce paths that approximately satisfy these constraints. In the first image of Fig. 2, the grey region illustrates the discretized representation of I , as each configuration is captured as an image frame, resulting in a finite set of $\mathbb{K}$ sampled at a fixed frame rate. The second image of Fig. 2 highlights similar discretized representation of I_{free} .

Given the above definitions, let us define the vision-only motion planning problem. *Vision-Only Motion Planning Problem:* Given the tuple $(I, RI, \mathcal{U}, \mathcal{IO}, Im, \mathbb{K}_{\text{start}}, I_g)$ find a time $\mathcal{T}$ and a set of controls $u : [0, \mathcal{T}] \rightarrow \mathcal{U}$ such that the motion described by g satisfies $\mathbb{K}(0) = \mathbb{K}_{\text{start}}$, $\mathbb{K}(\mathcal{T}) \in I_g$ and $\mathbb{K}(t) \in I_{\text{free}}$.

Algorithm 1: Build-Lazy-PRM.

```

1: procedure Build-Lazy-PRM,  $k$ 
2:    $G \leftarrow \text{INIT}()$ 
3:   while  $G.\text{size}() \leq N$  do
4:      $\mathbb{K}_{\text{new}} \leftarrow \text{SAMPLE}(I)$ 
5:      $G.\text{addNode}(\mathbb{K}_{\text{new}})$ 
6:     for each  $\mathbb{K} \in G.\text{nodes}()$  do
7:        $\mathcal{N}(\mathbb{K}) \leftarrow \text{K-NEAREST}(\mathbb{K}, G)$ 
8:       for each  $\mathbb{K}_{\text{near}} \in \mathcal{N}(\mathbb{K}_{\text{new}})$  do
9:          $e \leftarrow (\mathbb{K}, \mathbb{K}_{\text{near}})$ 
10:        if  $e \notin G.\text{edges}()$  then
11:           $G.\text{addEdge}(e)$ 
12:   return  $G$ 

```

Algorithm 2: Query-Lazy-PRM.

```

1: procedure Lazy-Query-PRM  $\mathbb{K}_{\text{start}}, \mathbb{K}_{\text{goal}}, G$ 
2:    $G.\text{add}(\mathbb{K}_{\text{start}}) \triangleright \text{Add start, and goal to the Graph}$ 
3:    $G.\text{add}(\mathbb{K}_{\text{goal}})$ 
4:    $\text{Edges}, \leftarrow \text{SEARCH-GRAPH } G(\mathbb{K}_{\text{goal}}, \mathbb{K}_{\text{start}})$ 
5:   for each  $e \in G.\text{edges}()$  do
6:     if  $\text{COLL\_FREE}(e)$  then
7:       for each  $\mathbb{K}_{\text{near}} \in \mathcal{N}(x_{\text{new}})$  do
8:          $e \leftarrow (\mathbb{K}, \mathbb{K}_{\text{near}})$ 
9:         if  $\text{COLL\_FREE}(e)$  and  $e \notin G.\text{edges}()$  then
10:         $G.\text{addEdge}(e)$ 
11:   return  $\text{Path}$ 

```

IV. METHODOLOGY

Since the system dynamics equation is unknown, we cannot directly plan in the control space $\mathcal{U}$. Instead, we will plan a path $\mathbb{K}_0, \mathbb{K}_1 \dots, \mathbb{K}_n$ directly in the I -space and then control the robot to follow the path with a vision-only controller [15].

To compute the path, which is the main contribution of this work, we propose to use probabilistic road-map planner (PRM) by adapting its subroutines to operate directly in I -space. Specifically, we opted to adapt the LAZY-PRM [33] planner due to its compatibility with our particular requirements. However, any sampling-based planner which relies on the same subroutines could be used.

Similar to PRM, LAZY-PRM operates in two phases: a building phase Algorithm 1, where the roadmap is constructed without collision checking, and a query phase Algorithm 2, where a path is searched and only the selected edges are checked for collisions. If any are non-viable, the roadmap is updated and the search is repeated. This separation allows the roadmap to be built offline, before robot deployment, without considering obstacles, enabling reuse across different environments and physical setups.

We use the probabilistic roadmap method for its efficiency in multi-query scenarios, where a single precomputed graph supports multiple start-goal pairs. The lazy variant is particularly suitable as the environment remains mostly unchanged between planning queries. Its ability to handle high-dimensional spaces also makes it well-suited for our I -space representation based on visual keypoints. We first describe how LAZY-PRM works, followed by the modifications needed to adapt it to I -space.

The building-phase (Algorithm 1) works with the following procedure. First, in line 4, a sample is generated and added in graph G . Then the k nearest neighbors are found by using a

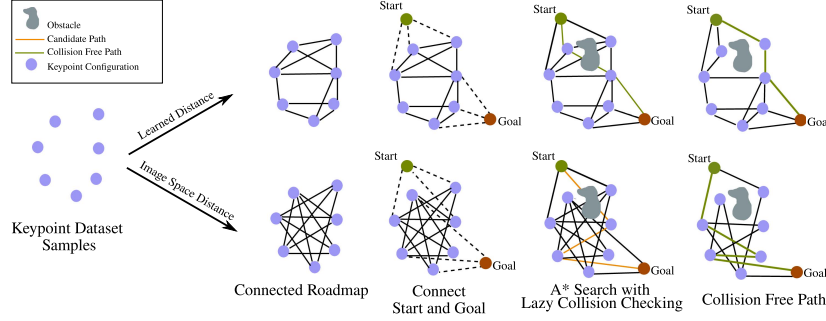


Fig. 3. Overview of the roadmap creation process for motion planning. In this image the purple circles are nodes in image space represented by image state in each frame. Using a distance metric, we connect the nodes to create a roadmap. As observed two different metrics produce different edges on the graph. After an A* search for path finding between a pair of start ($\mathbb{K}_{\text{start}}$) and goal ($\mathbb{K}_{\text{goal}}$) configurations and collision check if required, paths are found for both roadmaps. The green lines denote the final path after collision check. As observed, the **learned** distance roadmap has a clearly more optimized path than the **image space** distance roadmap.

distance defined in I (line 7) and are connected with edges. This continues until N nodes are in the graph G .

During the query-phase (Algorithm 2) a new motion planning problem is solved. Given a $\mathbb{K}_{\text{start}}$ and $\mathbb{K}_{\text{goal}}$ they are added in the graph, and connected with their nearest-neighbors. Then a graph search algorithm e.g., A* is used to find a path. If edges of the path are in-collision they are updated accordingly in the roadmap, and the process repeats until a collision free-path is found. The three operations SAMPLE, K-NEAREST, COLL_FREE, for Algorithm 1 in lines 4, 7 and Algorithm 2 typically require a model for the robot.

SAMPLE: This function typically samples the configuration space uniformly. However, in the vision-only setting, we can't directly sample in I -space as we don't have the model of the robot. Since $\dim(I) > \dim(\mathcal{X})$ the keypoint vectors (image state $\mathbb{K}$) that correspond to actual configurations of the robot, will lie in a lower-dimensional (equal to $\dim(\mathcal{X})$) manifold in I -space. Thus randomly sampling I -space will have 0 probability of sampling a valid configuration that lies on the valid manifold [23]. We describe how to address this issue in Section IV-A, by collecting and storing valid samples directly from the real robot.

K-NEAREST: This function usually relies on a distance defined in $\mathcal{X}$ and finds the nearest configurations that can be connected. However, since our representation in I -space is now a non-holonomic system, defining this distance is very challenging, as a straight line path in I -space defined by a simple Euclidean distance, might not be accurately followable by a controller. To mitigate this, we describe a learning-based approach to estimate the unknown joint-distance, in Section IV-B.

COLL_FREE: This function checks if there is a collision for an edge in $\mathcal{X}$. Again this typically requires the model $R(x)$ for the robot. In the vision only case, we propose simple yet successful method to do collision checking with an $RI(x)$ directly in I -space Section IV-C.

In the next section we describe our proposed method for each of the aforementioned subroutines. Fig. 3 visually describes all the steps of the visual motion planning framework.

A. SAMPLE: Executed Trajectories as Proxy Samples

In the absence of a robot model, we represent each image state $\mathbb{K} \in I$ by identifying and annotating keypoints ($\mathbb{k}$) on the robotic arm using an automated data collection pipeline described in [16], [17]. Each $\mathbb{K}_n$ is composed of a set of N

keypoints $\mathbb{k}$, where each $\mathbb{k}$ represents a pixel point of a specific location on the robot's body in image space. For instance, if $N = 5$, a keypoint vector or image state $\mathbb{K}_n$ is represented by a vector of $\mathbb{k}_1, \mathbb{k}_2, \mathbb{k}_3, \mathbb{k}_4, \mathbb{k}_5$, where each $\mathbb{k}$ is a pixel point in the n^{th} image frame. An example vector of such keypoints is shown in Fig. 2 as $\mathbb{K}_{\text{start}}$ or $\mathbb{K}_{\text{goal}}$.

To systematically explore the robot's visible workspace in the image space and ensure comprehensive coverage, we compute velocities for each joint j using:

$$v_j = \min \left(\frac{M_j}{\text{res} \cdot dt}, v_{\max} \right), \quad (5)$$

where, M_j is the motion range or difference of limits of joint j , res , is the number of discrete steps used to traverse M_j , dt is the duration allocated to complete each step, and $v_{\max}$ is the maximum allowable velocity for joint j . By dividing the motion range of each joint into uniform increments based on res , we ensure that the robot systematically explores all possible configurations within its workspace. The resulting dataset of image states $\mathbb{K}$ is thus evenly distributed across the image space I , enabling robust coverage and accurate representation of the robot's motion capabilities.

Each configuration $\mathbb{K}_n$ is computed using the following transformation:

$$\mathbb{K}_n = K \cdot T_{cw} \cdot x_n \quad (6)$$

where K is the camera intrinsic matrix, T_{cw} is the camera extrinsics matrix derived from calibration processes described in [34], [35], and x_n is the 3D robot configuration in workspace. This transformation projects the 3D configuration x_n , into their corresponding 2D (pixel) projection in the image described in [36], resulting in a set of $\mathbb{k}$ for each image state $\mathbb{K}_n$. The process captures the robot's motion across its visible workspace and creates a comprehensive dataset of $\mathbb{K}$, representing close to all feasible configurations in image space. Dataset of $\mathbb{K}$ can also be collected by following the process describe in [17].

The **Keypoint Dataset Samples** section of Fig. 3, represents this part of the workflow. The grey area in the left image of Fig. 2 illustrates how $\mathbb{K}$ are distributed within image space with each frame consisting of a set of $\mathbb{k}$ or keypoints. This collection process yields a dataset of $\mathbb{K}$ that can be used to enable efficient roadmap construction for vision-based motion planning.

Since this data collection process relies only on observable image features, it can be easily applied to other robotic systems, including soft and rigid manipulators, enabling broad applicability of the proposed framework.

B. K-NEAREST: Learned and Image Space Metrics

To search for the K-nearest neighbors of each image state sample we employ the following two distance metrics:

- *Learned distance*, where the distance is learned by a neural network, trained to predict joint displacements between two image states. The input to the network is a pair of image states $\mathbb{K}_1$ and $\mathbb{K}_2$ and the output is an estimated joint displacement required to transition between them:

$$\text{dist}_{\text{learned}}() \leftarrow \text{NN}(\mathbb{K}_1, \mathbb{K}_2) \quad (7)$$

- *Distance in Image space*, where we simply calculate the Euclidean distance between $\mathbb{K}_1$ and $\mathbb{K}_2$ in image space:

$$\text{dist}_{\text{image}}() \leftarrow \|\mathbb{K}_1 - \mathbb{K}_2\|_2 \quad (8)$$

Each metric influences the graph's structure by determining the nearest neighbors and defining the edges in the roadmap. The learned distance prioritizes image states with minimal estimated joint displacement, while the image space distance favors image states that are closer in the image. These differences impact the connectivity of the roadmap, as illustrated in the **Connected Roadmap** and **Connect Start and Goal** sections of Fig. 3.

1) *Dataset Generation for Network Model*: While collecting the dataset of image states $\mathbb{K}$ in Section IV-A, we recorded the velocity (5) applied to transition between consecutive frames. Using this data, we created a new dataset that includes pairs of consecutive image states ($\mathbb{K}_1, \mathbb{K}_2$) and their estimated joint displacements. This is calculated by multiplying the recorded velocity with the frame rate at which $\mathbb{K}$ was captured, as described in (6).

Only pairs of $\mathbb{K}$ captured in consecutive image frames are included in this data generation process. However, constructing the graph G , requires computing distances between arbitrary $\mathbb{K}$ pairs in I -space. To achieve this, a neural network is trained to predict joint displacements between different image-state pairs $\mathbb{K}$.

To enhance diversity, the dataset is augmented by combining frame sequences where the first frame's image state ($\mathbb{K}_{\text{start}}$) and the last frame's image state ($\mathbb{K}_{\text{end}}$) act as boundaries, with total joint displacements calculated as sum of displacements across intermediate frames. This approach ensures diverse transitions, enabling the neural network to accurately estimate joint displacements for any image state pair.

2) *Neural Network for Learned Distance Metric*: To derive (7), we design a simple neural network using the aforementioned dataset to learn a distance more similar to the **joint space** distance. The model takes concatenated arrays of the starting image state ($\mathbb{K}_{\text{start}}$) and the subsequent image state ($\mathbb{K}_{\text{next}}$) as input and predicts the estimated joint displacement. We train the network using the Adam optimizer (learning rate: 0.005), a batch size of 32, and 400 epochs, minimizing mean squared error (MSE) loss. Performance is evaluated using RMSE, MAE, and R^2 metrics.

C. COLL_FREE: image-Based Collision Checking

To enable collision checking in our model-free system, we employ an image-based polygon collision-checking framework. Obstacles are defined directly in the image space by drawing contours around visible objects and expanding them with a safety margin to account for the robot's volume and controller uncertainty. Each candidate path is checked by testing for intersection between the expanded obstacle region and polygons formed by corresponding keypoint pairs across two image states. This method effectively handles obstacles of arbitrary shape

TABLE I
ROADMAP BUILDING AND AVERAGE PATH GENERATION TIME (S)

	Joint Space	Learned	Image Space
Roadmap creation time (s)	0.25	766.19	0.28
Query path time (s)	2.422	3.19	2.38

and size and does not require access to the robot's model or workspace geometry. The collision-checking process and A* search is shown in **A* Search with Lazy Collision Checking** of Fig. 3.

D. Adaptive Visual Servoing

This work builds on the adaptive visual servoing method described in [15], using a roadmap of collision-free sequence of goal image states. At each goal, vector of keypoints ($\mathbb{k}$) in image state ($\mathbb{K}$) is tracked as visual features as described in [16]. The controller moves the arm minimizing the feature error, computed as the difference between the current and the target $\mathbb{K}$. The Jacobian matrix, estimated online via least-square optimization of recent joint velocities and keypoints vector over a moving window, eliminates the need to read joint position from encoder. This makes the control pipeline completely model-free. To improve accuracy, we reset the Jacobian estimation window at each new target keeping the estimate unbiased and relevant to the current goal. Since goal spacing in the image may vary across experiments, a saturation limit prevents velocity spikes, ensuring smooth motion and protecting the motor.

V. EXPERIMENTS AND OBSERVATIONS:

We experimentally assessed the performance of the proposed vision only motion planning framework on a Franka Emika Panda Arm [37].

A. Experimental Details

1) *Generating Samples*: We generated the required image state samples by collecting a finite dataset created by actuating the planar joints (Joints 2,4,6) of the Franka arm, using velocities computed from (5) as explained in Section IV-A, covering the robot's planar workspace.

2) *Building the Roadmap*: The collected samples $\mathbb{K}$ were used as nodes to build roadmaps based on the distance metrics described in Section IV-B. We refer to the roadmap using image space distances as the **Image Space** roadmap, and the one using learned distances as the **Learned** roadmap. For benchmarking, we also construct a **Joint Space** roadmap using actual joint displacements from encoders, used only for comparison, while preserving the model-free design. The k-neighbor value for all approaches was set to 25. The first row of Table I shows the time required to generate each roadmap. The **learned** roadmap takes significantly longer to construct due to the additional computational overhead of running neural network inference during nearest neighbor computation.

3) *Obstacle Representation and Path Planning*: In the experimental setup, obstacles were modeled as virtual shapes such as a rectangle or a triangle or a circle with a safety margin, as shown in the first image of Fig. 4. Paths were generated offline for various start ($\mathbb{K}_{\text{start}}$) and goal ($\mathbb{K}_{\text{goal}}$) incorporating the obstacle avoidance logic from Section IV-C. These paths were later used in adaptive visual servoing experiments.

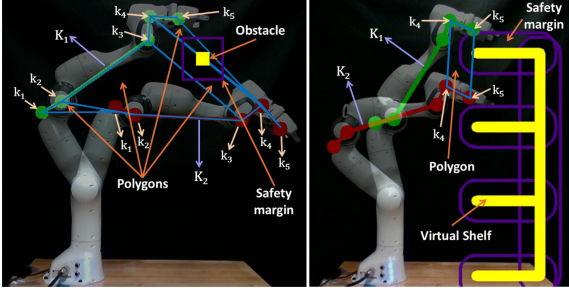


Fig. 4. Polygon-based collision checking in image space. Four polygons (light blue) are defined by connecting consecutive keypoints— (k_1, k_2) , (k_2, k_3) , (k_3, k_4) , and (k_4, k_5) —from the current image state (k_1 , green) to its neighbor (k_2 , red). A collision is detected if any polygon intersects the obstacle contour (yellow) inflated by a purple safety margin, as seen for $([k_2, k_3]$, $[k_3, k_4]$, and $[k_4, k_5]$) in the first image and $[k_4, k_5]$ in the shelf scenario.

4) *Control Experiment Set-Up for Single Obstacle*: The real-time control experiments used an Intel Realsense D435i camera in an eye-to-hand setup for visual feedback. The Panda arm followed the planned paths in 16 obstacle-free and 10 obstacle-avoidance experiments. The performance of each proposed roadmap was evaluated by comparing the joint position changes between intermediate image states to those of the **joint space** roadmap. The controller's ability to guide the arm along collision-free paths was evaluated for efficiency and effectiveness.

5) *Control Experiment Set-Up for Multiple Obstacles*: To further evaluate the robustness of our planning method, we conducted 4 additional experiments involving scenes with multiple obstacles of varying shapes and sizes. The **learned** roadmap successfully generated feasible paths in all 4 scenarios. In contrast, the **image space** roadmap was able to find valid paths in only 2 out of 4 cases, failing in the remaining two due to limitations in its Euclidean distance metric when navigating tighter environments.

6) *Control Experiment Set-Up for Virtual Shelf*: We conducted a proof-of-concept experiment simulating a real-world object (a multi-level shelf), as shown in the second image of Fig. 4, and planned paths using both the **learned** and **image space** roadmaps.

B. Roadmap and Path Planning Experiments

In this section, we analyze the joint displacements along the edges of the three roadmaps to evaluate their efficiency. Path planning experiments were conducted in both collision-free and obstacle-avoidance scenarios to compare the joint distances covered by the paths generated from each roadmap.

1) *Distribution of Joint Distances of Edges for Different Roadmaps*: Fig. 5 shows the joint displacement histograms for the three roadmaps. The **learned** roadmap closely matches the **joint space** roadmap, with only minor deviations, indicating accurate estimation of joint displacements. In contrast, the **image space** roadmap shows a wider spread and larger displacements, suggesting less efficient transitions.

2) *Average Joint Distances for Planned Collision-Free Paths*: We randomly selected 100 k_{start} and k_{goal} pairs from the roadmaps for path planning without obstacles. As shown in Table II, the **learned** roadmap produces joint distances closer to the **joint space** baseline, while the **image space** roadmap results in significantly higher displacements. This indicates that

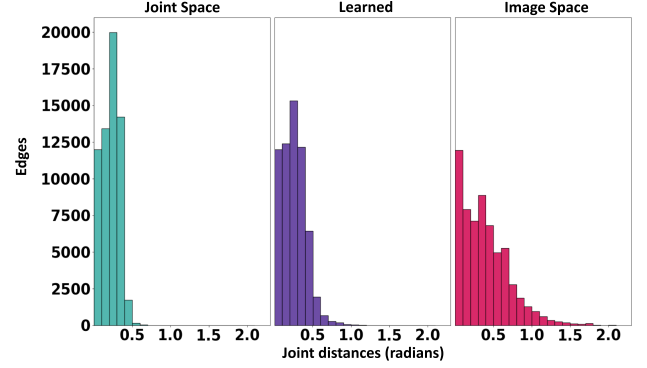


Fig. 5. Distribution of joint displacements along roadmap edges. The **Learned** roadmap closely aligns with the **Joint Space** distribution, indicating accurate distance estimation. In contrast, the **Image Space** roadmap shows a wider spread, reflecting poor alignment between image and joint space.

TABLE II
AVERAGE JOINT DISTANCES (RADIAN) FOR COLLISION-FREE PATHS

	Joint Space	Learned	Image Space
Mean (radians)	1.74	2.19	3.06

TABLE III
COMPARISON OF JOINT DISTANCES AND KEYPOINTS DISTANCES IN IMAGE SPACE OVER EXPERIMENTS

Roadmaps	Without Obstacle Avoidance			With Obstacle Avoidance		
	Joint Space	Learned	Image Space	Joint Space	Learned	Image Space
Number of Experiments	16	16	16	10	10	10
Avg. No. of Waypoints	8	8	14	11	13	15
Avg. Joint Distances (radians) b/w Waypoints	0.25	0.3	0.32	0.28	0.28	0.35
Avg. Keypoints Distances (pixels) b/w Waypoints	167.99	174.76	84.11	139.53	129.13	89.36
Avg. Joint Distances (radians) over Entire Path	1.92	2.27	4.29	3.04	3.46	5.36
Avg. Keypoints Distances (pixels) over Entire Path	1222.42	1278.92	1157.00	1532.72	1569.37	1361.43
Joint Distance (radians) Traversed To Move 1000 pixels in image space	1.6	1.77	3.79	1.98	2.19	3.9

the **learned** roadmap generates more efficient paths, better suited for control convergence.

3) *Planned Paths for Control Experiments*: We generated planned paths for two scenarios: 16 start and goal pairs without checking for collision and 10 pairs with collision avoidance. Paths were computed using the three roadmaps: **joint space**, **learned**, and **image space**. These precomputed paths were used in the control experiments described in Section V-C. The second row of Table I reports the average time each roadmap took to generate paths for these scenarios.

For each planned path, we calculated metrics including the average number of waypoints (intermediate image states), joint distances between waypoints, total joint distances for the entire path, and Euclidean distances between image states (keypoint distances) both between waypoints and across the entire path.

As observed in Table III the **learned** roadmap consistently resulted in fewer waypoints and shorter joint distances compared

TABLE IV
PERFORMANCE RESULTS FOR THE CONTROL EXPERIMENTS WITH AND WITHOUT OBSTACLE AVOIDANCE

Performance Metrics	Without Obstacle Avoidance			With Obstacle Avoidance		
	Joint Space	Learned	Image Space	Joint Space	Learned	Image Space
Successful Experiments	16/16	16/16	13/16	10/10	10/10	5/10
System Rise time (s)	74.92	97.53	94.75	105.38	125.38	90.46
System Settling time (s)	94.37	118.62	101.99	118.18	155.14	96.44
End effector Rise time (s)	74.86	97.53	94.39	105.02	125.38	90.46
End effector Settling time (s)	94.37	114.02	99.81	118.18	147.14	94.22
Overshoot (%)	1.94	2.61	1.89	1.93	2.36	1.35
Execution time (s)	125.17	148.92	140.78	146.02	201.68	124.70

to the **image space** roadmap, which prioritizes minimizing keypoints distances in image space but incurs higher joint displacements.

Notably, the joint distances required to traverse 1000 pixels in image space were much higher for the **image space** roadmap than for the **learned** roadmap. This suggests that reliance on **image space** proximity may lead to less efficient joint-space paths.

We have two theories from the above observations:

- The **image space** roadmap uses Euclidean distances, leading A* to favor shorter pixel paths with more intermediate waypoints. In contrast, the **learned** roadmap, based on joint displacements, yields more direct paths with fewer waypoints.
- Image states that are close in pixel space can be far apart in **joint space**, as shown in Table III. This can result in larger joint movements and control errors, especially in systems with non-holonomic constraints.

C. Control Experiments

The adaptive visual servoing experiments² used precomputed paths from Section V-B3, with control gains optimized to minimize rise and settling times while keeping overshoot within 5%, by careful tuning. We also performed adaptive visual-servoing experiments for the 4 scenarios for **learned** and **image space** roadmaps as defined in Section V-A5. To highlight the advantages of planning-based control over reactive methods, we implemented an Artificial Potential Field (APF) controller [38] and tested it on the same 4 multi-obstacle scenarios from Section V-A5.

The control metric in Table IV shows that the **image space** roadmap succeeded in 69.2% of cases, while the **learned** roadmap achieved 100% success. When successful, the **image space** roadmap had faster transients. This can be attributed to its shorter pixel-distance paths, which reduce image space error more quickly in scenarios where joint displacements are relatively smaller. Table V uses the same metrics as Table III, grouped by success and failure.

Image space roadmaps fail when the robot tries to follow short pixel paths that require large joint movements. Due to non-holonomic constraints, the robot might need to traverse a curved path in **joint space** to achieve even a small shift in **image space**, risking collisions or workspace violations. **Learned** roadmaps

²Planning and control experiments including apf videos, are available at this link. The details of how to use the link is in the supplementary ReadMe file. The supplementary video and relevant codes can be found in respective links

TABLE V
COMPARISON OF JOINT DISTANCES AND KEYPOINTS DISTANCES IN IMAGE SPACE OVER EXPERIMENTS FOR SUCCESSFUL AND FAILED EXPERIMENTS

Roadmaps	Successful			Failed (Image Space)		
	Joint Space	Learned	Image Space	Joint Space	Learned	Image Space
Number of Experiments	18	18	18	8	8	8
Avg No. of Waypoints	8	8	14	11	13	15
Avg. Joint Distances (radians) b/w Waypoints	0.25	0.3	0.32	0.27	0.29	0.35
Avg. Keypoints Distances (pixels) b/w Waypoints	166.77	163.41	86.03	135.17	143.24	86.35
Avg. Joint Distances (radians) over Entire Path	2.10	2.37	4.42	2.92	3.52	5.33
Avg. Keypoints Distances (pixels) over Entire Path	1307.02	1257.02	1210.93	1419.93	1691.26	1291.2
Joint Distance (radians) Traversed To Move 1000 pixels in image space	1.6	1.9	3.71	2.1	2.1	4.12

TABLE VI
PERFORMANCE COMPARISON OF LEARNED AND IMAGE SPACE ROADMAPS AND APF WITH MULTIPLE OBSTACLES

Roadmap	Successful Planning.	Successful Control.	System Rise Time (s)	System Settling Time (s)	EE Rise Time (s)	EE Settling Time (s)	Overshoot (%)	Exec. Time (s)
Learned	4/4	4/4	96.53	99.68	95.98	97.83	0.87	121.65
Image Space	2/4	2/2	89.1	102.6	89.1	100.05	0.43	122.45
APF	NA	0	NA	NA	NA	NA	NA	NA

TABLE VII
PERFORMANCE COMPARISON FOR LEARNED AND IMAGE SPACE ROADMAPS WITH PROOF-OF-CONCEPT REAL-OBJECT SCENARIO

Roadmap	Successful Planning.	Successful Control.	System Rise Time (s)	System Settling Time (s)	EE Rise Time (s)	EE Settling Time (s)	Overshoot (%)	Exec. Time (s)
Learned	1/1	1/1	34.8	75.8	28.9	75.8	3.24	97.5
Image Space	1/1	0/1	NA	NA	NA	NA	NA	NA

avoid this by producing motions that better match the robot's capabilities.

Table VI depicts the performance of **learned** and **image space** roadmaps in multi-obstacle scenarios. While **image space** failed to plan in 50% of the cases, all successful plans led to successful control; the **learned** roadmap succeeded in both planning and control for all cases. The APF controller failed to converge in all scenarios.

Despite its simplicity, APF failed in all cases, where the robot started between closely spaced obstacles, getting trapped in local minima and oscillating between attractive and opposing repulsive forces, without reaching the goal. These failures highlight the limitations of reactive methods in cluttered scenes and the necessity of global planners like our roadmap-based approach for collision avoidance.

Table VII presents control performance in a proof-of-concept scenario involving simulated real-world object, as described in Section V-A6. Here, the path planned by the **image space** roadmap led to a collision with the object, despite a defined safety margin. Further experiments are needed to conclusively determine why the robot collided despite a collision-free planned path.

In summary, both roadmaps exhibit unique benefits. The **image space** roadmap enables faster execution when successful but lacks reliability, while the **learned** roadmap offers greater

robustness by leveraging joint displacement-like distances. The choice depends on the application context.

VI. CONCLUSION AND FUTURE WORK

In conclusion, this work introduced a novel framework for collision-free motion planning of robotic manipulators that relied solely on visual features, eliminating the need for explicit robot models or encoder feedback. The **learned** roadmap offered smoother, more reliable transitions, and due to its joint displacement-based distance definition, the paths it generated maintained joint-space holonomy when it existed. In contrast, the paths produced by the **image space** roadmap sometimes failed to maintain joint-space holonomy, even when holonomy existed in the **image space**. However, the **image space** roadmap provided faster transient responses and simplicity, making it advantageous for applications where speed and computational efficiency were prioritized.

As a future direction, to extend our approach to higher DoF and out-of-plane motion, we plan to reuse the same data collection pipeline, existing data collection pipeline while capturing trajectories that span the full 3D workspace. To ensure safe interaction with real-world obstacles, we plan to estimate monocular depth replacing the 2D safety polygon with a 3D bounding box. To improve keypoint robustness under environmental and self-occlusion, we aim to integrate a graph-based model to enforce spatial consistency within the keypoint detection network. We are also exploring the extension of this pipeline to soft and continuum robotic systems, specifically a soft origami arm. Additionally, We are also investigating a quantitative formulation to characterize and analyze the role of non-holonomic constraints in image-based motion planning.

REFERENCES

- [1] S. Hutchinson, G. D. Hager, and P. I. Corke, "A tutorial on visual servo control," *IEEE Trans. Robot. Autom.*, vol. 12, no. 5, pp. 651–670, Oct. 1996.
- [2] K. Hashimoto, "A review on vision-based control of robot manipulators," *Adv. Robot.*, vol. 17, no. 10, pp. 969–991, 2003.
- [3] B. Calli and A. M. Dollar, "Vision-based precision manipulation with underactuated hands: Simple and effective solutions for dexterity," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 1012–1018.
- [4] H. Cuevas-Velasquez, N. Li, R. Tylecek, M. Saval-Calvo, and R. B. Fisher, "Hybrid multi-camera visual servoing to moving target," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 1132–1137.
- [5] P. Ardón, M. Dragone, and M. S. Erden, "Reaching and grasping of objects by humanoid robots through visual servoing," in *Proc. EuroHaptics*, 2018, pp. 353–365.
- [6] J. Lai, K. Huang, B. Lu, and H. K. Chu, "Toward vision-based adaptive configuring of a bidirectional two-segment soft continuum manipulator," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatron.*, 2020, pp. 934–939.
- [7] M. Luo et al., "Orisnake: Design, fabrication, and experimental analysis of a 3-D origami snake robot," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1993–1999, Jul. 2018.
- [8] P. Liu, M. N. Huda, L. Sun, and H. Yu, "A survey on underactuated robotic systems: Bio-inspiration, trajectory planning and control," *Mechatronics*, vol. 72, p. 102443, 2020.
- [9] A. Gandhi, S.-S. Chiang, C. D. Onal, and B. Calli, "Shape control of variable length continuum robots using clothoid-based visual servoing," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 6379–6386.
- [10] I. Chavdarov, V. Nikolov, B. Naydenov, and G. Boiadjev, "Design and control of an educational redundant 3D printed robot," in *Proc. Int. Conf. Softw., Telecommun. Comput. Networks*, 2019, pp. 1–6.
- [11] C. D. Onal, M. T. Tolley, R. J. Wood, and D. Rus, "Origami-inspired printed robots," *IEEE/ASME Trans. Mechatron.*, vol. 20, no. 5, pp. 2214–2221, Oct. 2015.
- [12] M. A. M. Adzeman, M. H. M. Zaman, M. F. Nasir, M. Faisal, and S. M. M. Ibrahim, "Kinematic modeling of a low cost 4 DoF robot arm system," *Int. J. Emerg. Trends Eng. Res.*, vol. 8, 2020.
- [13] H. Wang, B. Yang, J. Wang, X. Liang, W. Chen, and Y.-H. Liu, "Adaptive visual servoing of contour features," *IEEE/ASME Trans. Mechatron.*, vol. 23, no. 2, pp. 811–822, Apr. 2018.
- [14] D. Navarro-Alarcon and Y.-H. Liu, "Fourier-based shape servoing: A new feedback method to actively deform soft objects into desired 2-D image contours," *IEEE Trans. Robot.*, vol. 34, no. 1, pp. 272–279, Feb. 2018.
- [15] A. Gandhi, S. Chatterjee, and B. Calli, "Skeleton-based adaptive visual servoing for control of robotic manipulators in configuration space," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 2182–2189.
- [16] S. Chatterjee, A. C. Karade, A. Gandhi, and B. Calli, "Keypoints-based adaptive visual servoing for control of robotic manipulators in configuration space," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 6387–6394.
- [17] S. Chatterjee, D. Doan, and B. Calli, "Utilizing inpainting for training keypoint detection algorithms towards markerless visual servoing," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 3086–3092.
- [18] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Automat.*, vol. 12, no. 4, pp. 566–580, Aug. 1996.
- [19] A. Candra, M. A. Budiman, and K. Hartanto, "Dijkstra's and A-star in finding the shortest path: A tutorial," in *Proc. Int. Conf. Data Sci., Artif. Intell. Bus. Analytics*, 2020, pp. 28–32.
- [20] J. Schulman et al., "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [21] Z. Zhao et al., "A survey of optimization-based task and motion planning: From classical to learning approaches," *IEEE/ASME Trans. Mechatron.*, early access, Oct. 4, 2024, doi: [10.1109/TMECH.2024.3452509](https://doi.org/10.1109/TMECH.2024.3452509).
- [22] A. Orthey, C. Chamzas, and L. E. Kavraki, "Sampling-based motion planning: A comparative review," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 7, 2024.
- [23] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 1, pp. 159–185, 2018.
- [24] M. Likhachev, G. J. Gordon, and S. Thrun, "Ara*: Anytime A* with provable bounds on sub-optimality," in *Proc. NIPS*, 2003.
- [25] B. J. Cohen, S. Chitta, and M. Likhachev, "Search-based planning for manipulation with motion primitives," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2010, pp. 2902–2908.
- [26] S.-O. Park, M. C. Lee, and J. Kim, "Trajectory planning with collision avoidance for redundant robots using Jacobian and artificial potential field-based real-time inverse kinematics," *Int. J. Control Autom. Syst.*, vol. 18, pp. 2095–2107, 2020.
- [27] W. Liu, H. Niu, M. N. Mahyuddin, G. Herrmann, and J. Carrasco, "A model-free deep reinforcement learning approach for robotic manipulators path planning," in *Proc. IEEE 21st Int. Conf. Control, Automat. Syst.*, 2021, pp. 512–517.
- [28] D. Zhou, R. Jia, H. Yao, and M. Xie, "Robotic arm motion planning based on residual reinforcement learning," in *Proc. IEEE 13th Int. Conf. Comput. Automat. Eng.*, 2021, pp. 89–94.
- [29] B. Ichter and M. Pavone, "Robot motion planning in learned latent spaces," *IEEE Robot. Automat. Lett.*, vol. 4, no. 3, pp. 2407–2414, Jul. 2019.
- [30] M. Zeller, R. Sharma, and K. Schulten, "Vision-based motion planning of a pneumatic robot using a topology representing neural network," in *Proc. IEEE Int. Symp. Intell. Control*, 1996, pp. 7–12.
- [31] Y. Mezouar and F. Chaumette, "Path planning in image space for robust visual servoing," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2000, pp. 2759–2764.
- [32] D. Lee, H. Lim, and H. J. Kim, "Obstacle avoidance using image-based visual servoing integrated with nonlinear model predictive control," in *Proc. IEEE 50th Conf. Decis. Control Eur. Control Conf.*, 2011, pp. 5689–5694.
- [33] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2000, pp. 521–528.
- [34] T. E. Lee et al., "Camera-to-robot pose estimation from a single image," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 9426–9432.
- [35] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.
- [36] "Camera calibration and 3D reconstruction," [Online]. Available: https://docs.opencv.org/4.x/d9/d0c/group_calib3d.html
- [37] S. Haddadin, "The Franka Emika robot: A standard platform in robotics research," *IEEE Robot. Automat. Mag.*, vol. 31, no. 4, pp. 136–148, Dec. 2024.
- [38] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, 2005.